

Mémento React Native 2025

Formation 3 jours · Référence complète

Sommaire

#	Section	Thèmes couverts
1	Sommaire général	Répartition des modules par jour
2	Tableaux récapitulatifs	Comparatifs, cheat sheets, décisions
3	Composants de base	View, Text, Image, TextInput, Pressable
4	Hooks essentiels	useState, useEffect, useCallback, useMemo, useRef
5	Flexbox & Styling	Propriétés, patterns courants, StyleSheet
6	Gestion du State	useState, useReducer, Context, Redux, Zustand
7	Navigation	Expo Router, React Navigation, patterns
8	Réseau & Données	Fetch, Axios, TanStack Query, cache
9	Stockage local	AsyncStorage, MMKV, SecureStore, SQLite
10	API natives	Caméra, GPS, Notifications, Capteurs
11	Animations	Animated API, Reanimated 3, Moti
12	Formulaires	React Hook Form, Zod, accessibilité
13	Tests	Jest, Testing Library, Detox
14	Publication & OTA	EAS Build, Submit, Update
15	Bonnes pratiques	Performance, sécurité, erreurs fréquentes
16	Écosystème	Bibliothèques recommandées 2025

1 · Sommaire général de la formation

Jour	Modules	Thèmes principaux	TP
Jour 1	1 + 2	Cross-platform, New Architecture, ES6+, TypeScript, JSX, Hooks, Installation, Debug	Compteur, galerie, debug
Jour 2	3 + 4	Redux Toolkit, Zustand, Flux, FlatList, Flexbox, Expo Router, Reanimated 3	Tâches Redux, navigation, swipe cards
Jour 3	5 + 6 + 7	Formulaires, Fetch, MMKV, GPS, Caméra, Notifications, Tests, EAS, OTA	Formulaire multi-étapes, app recettes

Répartition des 7 modules

Module 1 – Introduction	→ Jour 1 (matin)
Module 2 – Bases React Native	→ Jour 1 (après-midi)
Module 3 – Architecture	→ Jour 2 (matin)
Module 4 – Interface	→ Jour 2 (après-midi)
Module 5 – Formulaires & Données	→ Jour 3 (matin)
Module 6 – API natives	→ Jour 3 (milieu)
Module 7 – Usages avancés	→ Jour 3 (après-midi)

2 · Tableaux récapitulatifs

2.1 — Équivalences Web → React Native

HTML / CSS	React Native	Notes
<div>	<View>	Conteneur universel
<p> , 	<Text>	Tout texte DOIT être dans <Text>
	<Image>	Dimensions obligatoires pour URLs réseau
<input>	<TextInput>	Props différentes (onChangeText vs onChange)
<button>	<Pressable> OU <TouchableOpacity>	Pas de <button> natif
<a>	<Link> (Expo Router)	Ou Linking.openURL
/	<FlatList> OU <SectionList>	Rendu virtuel
<form>	<View> + React Hook Form	Pas de <form> en RN
<select>	<Picker> (@react-native-picker/picker)	
<textarea>	<TextInput multiline>	
onClick	onPress	
className	style	Ou className avec NativeWind
display: flex	Flex par défaut	flexDirection est column par défaut
overflow: scroll	<ScrollView>	
position: fixed	position: 'absolute'	Relatif au parent

2.2 — Choisir sa solution de state

Situation	Solution
State local simple (formulaire, toggle)	useState
State local avec logique complexe	useReducer
Partage entre 2-3 composants proches	Lifting state up
Thème, langue (partagé globalement, peu fréquent)	Context API
State global léger (auth, préférences)	Zustand

Situation	Solution
Grande app entreprise, state complexe	Redux Toolkit
Données serveur (fetch, cache, mutations)	TanStack Query
API REST complète avec cache Redux	RTK Query
State atomique interdépendant	Jotai

2.3 — Choisir son stockage

Besoin	Solution	Sync	Chiffrement
Clés/valeurs simples, performances	MMKV	Synchrone	Optionnel
Compatibilité Expo Go	AsyncStorage	Async	✗
Tokens, données sensibles	SecureStore	Async	✓ natif
Données relationnelles	Expo SQLite	Async	✗
App offline-first complexe	WatermelonDB	Async	Optionnel
Cache requêtes réseau	TanStack Query persist	Async	✗

2.4 — Choisir sa navigation

Critère	Expo Router	React Navigation 7
Type	File-based (Next.js-like)	Composants déclaratifs
Setup	Intégré Expo, zéro config	Config manuelle
Deep linking	Automatique	Config nécessaire
TypeScript	Natif	Typage manuel
Web support	✓ natif	Partiel
Drawer + Tabs imbriqués	✓	✓
Recommandé	Nouveaux projets Expo	Contrôle maximal

2.5 — Choisir son animation

Besoin	Solution
Opacité, scale, translate simples	Animated API (natif)
Animations 60fps, gestes, worklets	Reanimated 3
Animations déclaratives simples	Moti
Transition de layout automatique	LayoutAnimation
Dessins, graphiques, effets GPU	React Native Skia
Animations Lottie (JSON AE)	lottie-react-native

2.6 — Performances — Quand mémoïser ?

Hook	Utiliser quand	Ne pas utiliser si
React.memo	Composant re-render souvent sans que ses props changent	Props changent à chaque render
useCallback	Fonction passée en prop à un memo ou dans un useEffect	Fonction utilisée uniquement en local
useMemo	Calcul coûteux (> 1ms) qui dépend d'une valeur	Calcul trivial (somme, string)

2.7 — Versions clés 2025

Technologie	Version stable	Note
React Native	0.79	New Architecture activée par défaut
Expo SDK	53	React Native 0.79, React 19
React	19	Concurrent, Actions, use()
TypeScript	5.x	Support natif
Hermes	intégré RN	Moteur JS par défaut
Redux Toolkit	2.x	RTK Query inclus
Reanimated	3.x	Worklets, Gesture Handler 2
Expo Router	4.x	Server Components expérimental

3 · Composants de base

View

```
import { View, StyleSheet } from 'react-native';

// Conteneur de base – flex column par défaut
<View style={styles.container}>
  {/* enfants */}
</View>

// SafeAreaView – respecte les encoches iOS et Android
import { SafeAreaView } from 'react-native-safe-area-context';
<SafeAreaView style={{ flex: 1 }}>
  {/* contenu protégé des encoches */}
</SafeAreaView>
```

Props utiles : style , onLayout , pointerEvents , accessible , testID

Text

```
import { Text } from 'react-native';

<Text
  style={styles.titre}
  numberOfLines={2}           // Tronquer après N lignes
  ellipsizeMode="tail"       // head | middle | tail | clip
  selectable                  // Permet la sélection/copie
  onPress={handlePress}      // Text cliquable
  accessibilityRole="header" // Accessibilité
>
  Contenu textuel
</Text>

// ⚠ Imbriquer des Text pour le style partiel
<Text>Normal <Text style={{ fontWeight: 'bold' }}>gras</Text> normal</Text>
```

Image

```
import { Image } from 'react-native';
import { Image as ExpoImage } from 'expo-image'; // Recommandé

// Image locale (bundlée)
<Image source={require('./assets/logo.png')} style={{ width: 100, height: 100 }} />

// Image réseau – dimensions OBLIGATOIRES
<Image source={{ uri: 'https://...' }} style={{ width: 200, height: 200 }}
  resizeMode="cover" // cover | contain | stretch | repeat | center
  onLoad={handleLoad}
  onError={handleError}
/>

// Expo Image – recommandé (cache, transitions, blurhash)
<ExpoImage
  source="https://..."
  style={{ width: 200, height: 200 }}
  contentFit="cover" // Équivalent resizeMode
  transition={300} // Animation de chargement en ms
  placeholder={{ blurhash: 'L6PZfSi_.AyE_3t7t7R**0o#DgR4' }}
/>
```

TextInput

```
import { TextInput } from 'react-native';

<TextInput
  value={valeur}
  onChangeText={({texte) => setValeur(texte)}
  placeholder="Saisir..."
  placeholderTextColor="#999"
  keyboardType="email-address"    // default | email-address | numeric | phone-pad | url
  autoCapitalize="none"           // none | sentences | words | characters
  autoCorrect={false}
  secureTextEntry={true}          // Mot de passe
  multiline={true}                // Multiligne
  numberOfLines={4}               // Hauteur (multiline)
  returnKeyType="next"            // next | done | search | send | go
  onSubmitEditing={handleSubmit}
  ref={inputRef}                  // Focus programmatique : inputRef.current?.focus()
  maxLength={100}
  editable={!changement}
/>
```

Pressable / TouchableOpacity

```
// Pressable – contrôle complet (recommandé)
<Pressable
  style={({ pressed }) => [
    styles.bouton,
    pressed && styles.boutonPresse,    // Style dynamique au press
  ]}
  onPress={handlePress}
  onLongPress={handleLongPress}
  delayLongPress={500}
  disabled={changement}
  hitSlop={{ top: 10, bottom: 10, left: 10, right: 10 }} // Zone tactile élargie
>
  {{{ pressed }} => (
    <Text style={{ opacity: pressed ? 0.7 : 1 }}>Appuyer</Text>
  )}
</Pressable>

// TouchableOpacity – plus simple
<TouchableOpacity onPress={handlePress} activeOpacity={0.7}>
  <Text>Appuyer</Text>
</TouchableOpacity>
```

FlatList

```
<FlatList
  data={items} // Tableau de données
  keyExtractor={({item}) => item.id} // ✅ Toujours un ID stable
  renderItem={({ item, index, separators }) => (
    <ItemRow item={item} />
  )}

  // Performance
  getItemLayout=({_, index}) => ({ // Hauteurs fixes = scroll fluide
    length: ITEM_HEIGHT,
    offset: ITEM_HEIGHT * index,
    index,
  })}
  windowSize={10}
  maxToRenderPerBatch={10}
  removeClippedSubviews={true} // Android uniquement

  // Mise en page
  numColumns={2} // Grille N colonnes
  horizontal // Défilement horizontal
  inverted // Chat (bas en haut)

  // Slots
  ListEmptyComponent={<TextVide />}
  ListHeaderComponent={<Entete />}
  ListFooterComponent={isLoading ? <ActivityIndicator /> : null}
  ItemSeparatorComponent={() => <View style={styles.separateur} />}

  // Pagination
  onEndReached={chargerPage}
  onEndReachedThreshold={0.3} // 30% avant la fin

  // Pull-to-refresh
  refreshing={refreshing}
  onRefresh={handleRefresh}
/>
```

4 · Hooks essentiels

useState

```
const [valeur, setValeur] = useState<string>('');
const [items, setItems] = useState<Item[]>([]);
const [user, setUser] = useState<User | null>(null);

// ✅ Mise à jour fonctionnelle (si nouvel état dépend de l'ancien)
setCount(prev => prev + 1);
setItems(prev => [...prev, nouvelItem]);
setUser(prev => ({ ...prev!, nom: 'Alice' }));

// ✅ Initialisation lazy (calcul coûteux exécuté une seule fois)
const [state, setState] = useState(() => computeInitialState());
```

useEffect

```
// Montage uniquement (tableau vide)
useEffect(() => {
  initialiser();
}, []);

// À chaque changement de `userId`
useEffect(() => {
  if (!userId) return;
  chargerUser(userId);
}, [userId]);

// Avec cleanup (abonnements, timers, requêtes annulables)
useEffect(() => {
  let actif = true; // Évite l'update si démonté
  const controller = new AbortController();

  fetch(url, { signal: controller.signal })
    .then(r => r.json())
    .then(data => { if (actif) setData(data); });

  return () => {
    actif = false;
    controller.abort(); // Annuler la requête
  };
}, [url]);
```

useCallback & useMemo

```
// useCallback – stabiliser une fonction (référence stable entre renders)
const handlePress = useCallback((id: string) => {
  dispatch(selectItem(id));
}, [dispatch]); // Dépendances

// useMemo – mémoriser un calcul
const total = useMemo(
  () => items.reduce((sum, i) => sum + i.prix * i.quantite, 0),
  [items]
);

const filtres = useMemo(
  () => items.filter(i => i.actif).sort((a, b) => b.date - a.date),
  [items]
);
```


useRef

```
import { useRef } from 'react';

// Référence à un composant natif (focus, measure...)
const inputRef = useRef<TextInput>(null);
inputRef.current?.focus();
inputRef.current?.blur();
inputRef.current?.clear();

// Valeur persistante entre renders (sans déclencher de re-render)
const intervalRef = useRef<ReturnType<typeof setInterval> | null>(null);
intervalRef.current = setInterval(() => { /* ... */ }, 1000);
// Cleanup :
clearInterval(intervalRef.current!);

// Valeur précédente
const prevCount = useRef(count);
useEffect(() => { prevCount.current = count; }, [count]);
```

5 · Flexbox & Styling

Propriétés Flexbox essentielles

Propriété	Valeurs	Défaut RN	Note
flexDirection	row , column , row-reverse , column-reverse	column	≠ Web (row)
justifyContent	flex-start , center , flex-end , space-between , space-around , space-evenly	flex-start	Axe principal
alignItems	flex-start , center , flex-end , stretch , baseline	stretch	Axe secondaire
alignSelf	Idem alignItems	auto	Sur l'enfant
flex	nombre	0	Part proportionnelle
flexWrap	nowrap , wrap , wrap-reverse	nowrap	
gap	nombre	0	RN 0.71+
rowGap / columnGap	nombre	0	Espacement précis

Patterns courants

```
// ① Centrer horizontalement et verticalement
style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}
```



```
// ② Header fixe + contenu flexible + footer fixe
<View style={{ flex: 1 }}>
  <View style={{ height: 56 }}>      {/* Header */}</View>
  <ScrollView style={{ flex: 1 }}>  {/* Contenu */}</ScrollView>
  <View style={{ height: 60 }}>      {/* Footer */}</View>
</View>
```



```
// ③ Grille 2 colonnes responsive
<FlatList numColumns={2} data={items} renderItem={...} />
```



```
// ④ Ligne avec icône à gauche, texte flexible, action à droite
<View style={{ flexDirection: 'row', alignItems: 'center', gap: 12 }}>
  <Icon size={24} />
  <Text style={{ flex: 1 }}>Texte</Text>
  <ChevronRight />
</View>
```



```
// ⑤ Badge absolu sur une icône
<View style={{ position: 'relative' }}>
  <Icon size={28} />
  <View style={{ position: 'absolute', top: -4, right: -4,
    width: 16, height: 16, borderRadius: 8, backgroundColor: 'red' }}>
    <Text style={{ color: 'white', fontSize: 10 }}>3</Text>
  </View>
</View>
```

StyleSheet — Bonnes pratiques

```
const styles = StyleSheet.create({
  // ✅ Utiliser StyleSheet.create (validation compile-time + optimisation)
  conteneur: {
    flex: 1,
    backgroundColor: '#fff',
  },

  // Ombres cross-platform
  carte: {
    backgroundColor: '#fff',
    borderRadius: 12,
    // iOS
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 8,
    // Android
    elevation: 4,
  },

  // ❌ À éviter – objet inline créé à chaque render
  // <View style={{ marginTop: 16 }} />
  // ✅ Préférer
  espaceur: { marginTop: 16 },
});
```

6 · Gestion du State

useState — Cas d'usage

```
// ✅ Adapté pour
const [modal, setModal] = useState(false); // Toggle UI
const [texte, setTexte] = useState(''); // Input contrôlé
const [page, setPage] = useState(1); // Pagination
const [theme, setTheme] = useState<'light'|'dark'>('light');

// ⚠️ Pas adapté si
// → Le state est partagé entre plusieurs écrans → Zustand/Redux
// → Le state dépend d'un serveur → TanStack Query
```

Redux Toolkit — Anatomie complète

```
// 1. Slice
const slice = createSlice({
  name: 'todos',
  initialState: [] as Todo[],
  reducers: {
    ajouter: (state, action: PayloadAction<string>) => {
      state.push({ id: uid(), text: action.payload, done: false }); // Immer OK
    },
  },
  extraReducers: (builder) => {
    // Gérer les thunks async
    builder.addCase(fetchTodos.fulfilled, (state, action) => {
      return action.payload;
    });
  },
});

// 2. Thunk async
export const fetchTodos = createAsyncThunk('todos/fetch', async () => {
  return await api.get<Todo[]>('/todos');
});

// 3. Sélecteur mémorisé
export const selectTodosActifs = createSelector(
  [(state: RootState) => state.todos],
  (todos) => todos.filter(t => !t.done)
);
```

Zustand — Template complet

```
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import { immer } from 'zustand/middleware/immer';
import AsyncStorage from '@react-native-async-storage/async-storage';

interface AppStore {
  user: User | null;
  token: string | null;
  login: (user: User, token: string) => void;
  logout: () => void;
}

const useAppStore = create<AppStore>()(
  persist(
    immer((set) => ({
      user: null,
      token: null,
      login: (user, token) => set({ user, token }),
      logout: () => set({ user: null, token: null }),
    })),
    {
      name: 'app-store',
      storage: createJSONStorage(() => AsyncStorage),
      partialize: (state) => ({ token: state.token }), // Ne persister que le token
    }
  )
);
```

7 · Navigation

Expo Router — Référence rapide

```
// Navigation
import { Link, useRouter, useLocalSearchParams, usePathname } from 'expo-router';

const router = useRouter();
router.push('/detail'); // Empiler
router.replace('/accueil'); // Remplacer sans historique
router.back(); // Retour
router.navigate('/tab'); // Intelligent (pop si déjà présent)

// Lien déclaratif
<Link href="/profil">Profil</Link>
<Link href={{ pathname: '/article/[id]', params: { id: '42' } }}>Article</Link>
<Link href="/modal" asChild> // Wrapper un composant custom
  <TouchableOpacity><Text>Modal</Text></TouchableOpacity>
</Link>

// Paramètres
const { id } = useLocalSearchParams<{ id: string }>();
const params = useGlobalSearchParams();
const path = usePathname(); // "/article/42"

// Gardes de navigation
import { Redirect } from 'expo-router';
if (!token) return <Redirect href="/(auth)/connexion" />;
```

Structure de fichiers — Patterns courants

```
app/
├─ _layout.tsx          → Provider globaux (Redux, QueryClient, ThemeProvider)
├─ index.tsx            → Accueil "/"
├─ +not-found.tsx       → 404
├─ (tabs)/              → Groupe tabs (invisible dans URL)
│   └─ _layout.tsx      → <Tabs> avec options
│   └─ index.tsx        → Onglet 1
│   └─ profile.tsx      → Onglet 2
├─ (auth)/              → Groupe auth
│   └─ login.tsx
│   └─ register.tsx
├─ (modals)/            → Modaux
│   └─ settings.tsx
└─ detail/
    └─ [id].tsx          → Route dynamique
```

8 · Réseau & Données

Pattern fetch robuste

```
// lib/fetcher.ts - wrapper universel
interface FetchOptions extends RequestInit {
  timeout?: number;
}

async function fetcher<T>(url: string, options: FetchOptions = {}): Promise<T> {
  const { timeout = 10_000, ...init } = options;
  const controller = new AbortController();
  const id = setTimeout(() => controller.abort(), timeout);

  try {
    const res = await fetch(url, { ...init, signal: controller.signal });
    if (!res.ok) {
      const err = await res.json().catch(() => ({}));
      throw new Error(err.message ?? `HTTP ${res.status}`);
    }
    return await res.json();
  } finally {
    clearTimeout(id);
  }
}
```

TanStack Query — Cheat sheet

```
// Query (lecture)
const { data, isLoading, isError, error, refetch, isFetching } = useQuery({
  queryKey: ['users', userId],          // Clé unique – tableau recommandé
  queryFn: () => api.getUser(userId),
  staleTime: 5 * 60 * 1000,            // Données fraîches pendant 5 min
  gcTime: 10 * 60 * 1000,              // Garder en cache 10 min (ex-cacheTime)
  retry: 3,                            // 3 tentatives si erreur réseau
  enabled: !!userId,                   // Ne pas lancer si userId est null
  select: (data) => data.items,         // Transformer les données
  placeholderData: keepPreviousData,   // Garder les données précédentes pendant le chargement
});

// Mutation (écriture)
const mutation = useMutation({
  mutationFn: (newTodo: NewTodo) => api.createTodo(newTodo),
  onMutate: async (newTodo) => {
    // Optimistic update
    await queryClient.cancelQueries({ queryKey: ['todos'] });
    const prev = queryClient.getQueryData(['todos']);
    queryClient.setQueryData(['todos'], old => [...old, { ...newTodo, id: 'temp' }]);
    return { prev }; // Context pour onError
  },
  onError: (err, _, context) => {
    queryClient.setQueryData(['todos'], context?.prev); // Rollback
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ['todos'] }); // Resync
  },
});

mutation.mutate({ text: 'Nouvelle tâche' });

// Infinite query (pagination)
const { data, fetchNextPage, hasNextPage } = useInfiniteQuery({
  queryKey: ['posts'],
  queryFn: ({ pageParam = 0 }) => fetchPosts({ page: pageParam }),
  getNextPageParam: (lastPage) => lastPage.nextCursor ?? undefined,
});
```


9 · Stockage local

AsyncStorage — Référence

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// CRUD
await AsyncStorage.setItem('key', JSON.stringify(value));
const raw = await AsyncStorage.getItem('key');
const value = raw ? JSON.parse(raw) : null;
await AsyncStorage.removeItem('key');
await AsyncStorage.clear(); // ⚠️ Tout supprimer

// Batch (plus efficace)
await AsyncStorage.multiSet([['k1', 'v1'], ['k2', 'v2']]);
const values = await AsyncStorage.multiGet(['k1', 'k2']);
await AsyncStorage.multiRemove(['k1', 'k2']);

const keys = await AsyncStorage.getAllKeys(); // Lister toutes les clés
```

MMKV — Référence (synchrone)

```
import { MMKV } from 'react-native-mmkv';

const storage = new MMKV({
  id: 'user-storage',
  encryptionKey: 'clé-secrète', // Chiffrement AES-256
});

// Synchrone — pas d'await !
storage.set('token', 'abc123');
storage.set('count', 42);
storage.set('data', JSON.stringify({ key: 'value' }));

const token = storage.getString('token');
const count = storage.getNumber('count');
const exists = storage.contains('token');

storage.delete('token');
storage.clearAll();

// Écouter les changements
storage.addOnValueChangedListener((key) => {
  console.log(`${key} a changé`);
});
```

10 · API natives

Géolocalisation

```
import * as Location from 'expo-location';

// Permission
const { status } = await Location.requestForegroundPermissionsAsync();
if (status !== 'granted') { /* Gérer le refus */ return; }

// Position unique
const pos = await Location.getCurrentPositionAsync({
  accuracy: Location.Accuracy.Balanced, // Low | Balanced | High | BestForNavigation
});
const { latitude, longitude, altitude, accuracy } = pos.coords;

// Suivi temps réel
const sub = await Location.watchPositionAsync(
  { accuracy: Location.Accuracy.High, TimeInterval: 5000, distanceInterval: 10 },
  (pos) => updateMap(pos.coords)
);
sub.remove(); // Cleanup

// Géocodage inverse (coords → adresse)
const [adresse] = await Location.reverseGeocodeAsync({ latitude, longitude });
console.log(adresse.city, adresse.street);
```

Notifications

```
import * as Notifications from 'expo-notifications';

// Configurer le comportement
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
  }),
});

// Notification immédiate
await Notifications.scheduleNotificationAsync({
  content: { title: 'Titre', body: 'Corps', data: { écran: 'profil' } },
  trigger: null,
});

// Notification différée
await Notifications.scheduleNotificationAsync({
  content: { title: 'Rappel' },
  trigger: { type: 'date', date: new Date('2025-12-01T08:00:00') },
});

// Notification récurrente
await Notifications.scheduleNotificationAsync({
  content: { title: 'Météo du matin' },
  trigger: { type: 'daily', hour: 8, minute: 0 },
});

// Écouter les réponses
Notifications.addNotificationResponseReceivedListener((response) => {
  const { écran } = response.notification.request.content.data;
  router.push(`/${écran}`);
});
```

Caméra & Galerie

```
// Caméra directe
import { CameraView, useCameraPermissions } from 'expo-camera';
const [permission, requestPermission] = useCameraPermissions();
// <CameraView ref={ref} style={{ flex: 1 }} facing="back" />
// const photo = await ref.current?.takePictureAsync({ quality: 0.8, base64: false });

// Galerie
import * as ImagePicker from 'expo-image-picker';
const result = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ['images'], // 'videos' | ['images', 'videos']
  allowsEditing: true,
  aspect: [1, 1],
  quality: 0.8,
});
if (!result.canceled) {
  const { uri, width, height, mimeType } = result.assets[0];
}
```

Haptics & Partage

```
import * as Haptics from 'expo-haptics';
// Retour tactile
await Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light); // Light | Medium | Heavy
await Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
await Haptics.selectionAsync();

import { Share, Linking } from 'react-native';
// Partage natif
await Share.share({ message: 'Texte à partager', url: 'https://...' });

// Ouvrir URL externe
await Linking.openURL('https://reactnative.dev');
await Linking.openURL('mailto:contact@example.com?subject=Bonjour');
await Linking.openURL('tel:+33612345678');
await Linking.openSettings(); // Paramètres de l'app
```

11 · Animations

Animated API — Référence

```
import { Animated, Easing } from 'react-native';

// Valeurs animées
const opacity = useRef(new Animated.Value(0)).current;
const translateY = useRef(new Animated.Value(-20)).current;

// Timing (durée fixe)
Animated.timing(opacity, {
  toValue: 1,
  duration: 400,
  easing: Easing.out(Easing.cubic),
  useNativeDriver: true, // ✅ Toujours pour opacity et transform
}).start((finished) => { /* callback */ });

// Spring (physique)
Animated.spring(translateY, {
  toValue: 0,
  tension: 100,
  friction: 8,
  useNativeDriver: true,
}).start();

// Séquence
Animated.sequence([anim1, anim2, anim3]).start();
// Parallèle
Animated.parallel([anim1, anim2]).start();
// Décalé
Animated.stagger(100, animations).start();
// Boucle infinie
Animated.loop(Animated.timing(...)).start();
```

Reanimated 3 — Référence

```
import Animated, {
  useSharedValue, useAnimatedStyle, useAnimatedProps,
  withTiming, withSpring, withRepeat, withSequence,
  interpolate, Extrapolation, runOnJS,
  FadeIn, FadeOut, SlideInLeft, ZoomIn, // Animations prédéfinies
} from 'react-native-reanimated';

// SharedValue - thread-safe
const progress = useSharedValue(0);

// Style animé (worklet - tourne sur UI thread)
const style = useAnimatedStyle(() => ({
  opacity: progress.value,
  transform: [
    { scale: interpolate(progress.value, [0, 1], [0.8, 1], Extrapolation.CLAMP) },
    { translateY: interpolate(progress.value, [0, 1], [-20, 0]) },
  ],
}));

// Animer
progress.value = withTiming(1, { duration: 400 });
progress.value = withSpring(1, { damping: 12, stiffness: 100 });
progress.value = withRepeat(withTiming(1, { duration: 800 }), -1, true);

// Appeler du code React depuis un worklet
const handleEnd = () => dispatch(nextItem());
// Dans le gestureHandler.onEnd :
runOnJS(handleEnd)();

// Layout animations - entrer/sortir
<Animated.View entering={FadeIn.duration(300)} exiting={FadeOut}>
<Animated.View entering={SlideInLeft.delay(200)}>
```

12 · Formulaires & Validation

React Hook Form + Zod — Template

```
import { useForm, Controller } from 'react-hook-form';
import { z } from 'zod';
import { zodResolver } from '@hookform/resolvers/zod';

// 1. Schéma
const schema = z.object({
  email: z.string().email('Email invalide'),
  password: z.string().min(8, 'Min. 8 caractères')
    .regex(/[A-Z]/, 'Une majuscule requise'),
  age: z.coerce.number().min(18, 'Majeur requis'),
});
type FormData = z.infer<typeof schema>;

// 2. Hook
const { control, handleSubmit, formState: { errors, isSubmitting }, reset, watch, setValue } =
  useForm<FormData>({
    resolver: zodResolver(schema),
    defaultValues: { email: '', password: '', age: 0 },
    mode: 'onChange', // Valider à chaque frappe (ou 'onBlur' | 'onSubmit')
  });

// 3. Composant champ réutilisable
function ChampTexte({ name, control, label, ...inputProps }) {
  return (
    <Controller
      control={control} name={name}
      render={({ field: { onChange, onBlur, value }, fieldState: { error } }) => (
        <View>
          <Text style={styles.label}>{label}</Text>
          <TextInput value={value} onChangeText={onChange} onBlur={onBlur} {...inputProps}
            style={[styles.input, error && styles.inputErreur]} />
          {error && <Text style={styles.erreur}>{error.message}</Text>}
        </View>
      )}
    />
  );
}

// 4. Soumission
const onSubmit = handleSubmit(async (data) => {
  await api.post('/auth/register', data);
});
```

Clavier & Accessibilité

```
import { KeyboardAvoidingView, Platform, TouchableWithoutFeedback, Keyboard } from 'react-native';

// Pattern universel pour formulaires avec clavier
<KeyboardAvoidingView
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
  style={{ flex: 1 }}
>
  <TouchableWithoutFeedback onPress={Keyboard.dismiss}>
    <ScrollView keyboardShouldPersistTaps="handled">
      {/* Champs du formulaire */}
    </ScrollView>
  </TouchableWithoutFeedback>
</KeyboardAvoidingView>
```

13 · Tests

Jest + Testing Library — Référence

```
import { render, fireEvent, waitFor, act } from '@testing-library/react-native';

// Queries – trouver des éléments
getByText('Valider')           // Lève si non trouvé
queryByText('Optionnel')       // Retourne null si non trouvé
findByText('Chargé...')        // Async – attend l'apparition

getByRole('button', { name: 'OK' })
getByPlaceholderText('Email')
getById('submit-button')       // testID="submit-button" sur le composant

// Actions
fireEvent.press(getByText('Valider'));
fireEvent.changeText(getByPlaceholderText('Email'), 'test@ok.com');
fireEvent.scroll(getById('liste'), { contentOffset: { x: 0, y: 100 } });

// Assertions
expect(getByText('Succès')).toBeTruthy();
expect(queryByText('Erreur')).toBeNull();
expect(getByText('Résultat')).toHaveStyle({ color: 'green' });

// Async
await waitFor(() => expect(getByText('Chargé')).toBeTruthy());
await act(async () => { fireEvent.press(btn); });

// Mock
jest.mock('../lib/api', () => ({ get: jest.fn().mockResolvedValue([]) }));
global.fetch = jest.fn().mockResolvedValue({
  ok: true,
  json: () => Promise.resolve({ data: [] }),
});
```

Test d'un reducer

```
// ✅ Les reducers sont des fonctions pures – faciles à tester
import reducer, { ajouter, supprimer } from '../store/todosSlice';

test('ajouter une tâche', () => {
  const state = reducer([], ajouter('Test'));
  expect(state).toHaveLength(1);
  expect(state[0].text).toBe('Test');
  expect(state[0].done).toBe(false);
});

test('ne mute pas le state original', () => {
  const initial = Object.freeze([]);
  expect(() => reducer(initial, ajouter('Test'))).not.toThrow();
});
```

14 · Publication & OTA

EAS — Flux de déploiement complet

```
# 1. Installer EAS CLI
npm install -g eas-cli
eas login

# 2. Configurer le projet
eas build:configure          # Crée eas.json

# 3. Secrets (variables d'environnement sécurisées)
eas secret:create --name API_URL --value "https://api.prod.com"
eas secret:list

# 4. Builds
eas build --platform android --profile preview    # APK de test
eas build --platform ios      --profile preview    # IPA de test
eas build --platform all      --profile production # Stores

# 5. Publication
eas submit --platform ios      # → App Store Connect
eas submit --platform android  # → Google Play Console

# 6. Mise à jour OTA (bundle JS uniquement)
eas update --branch staging     --message "Fix: crash liste"
eas update --branch production --message "v1.2.0"

# 7. Vérifier les mises à jour dans l'app
import * as Updates from 'expo-updates';
const update = await Updates.checkForUpdateAsync();
if (update.isAvailable) {
  await Updates.fetchUpdateAsync();
  await Updates.reloadAsync();
}
```


eas.json — Template

```
{
  "cli": { "version": ">= 10.0.0" },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" }
    },
    "production": {
      "autoIncrement": true
    }
  },
  "submit": {
    "production": {
      "ios": { "appleId": "dev@societe.com", "ascAppId": "123456789" },
      "android": { "serviceAccountKeyPath": "./service-account.json" }
    }
  }
}
```

Limites OTA

Changement	OTA possible ?
Correction de bug JS/TS	✔ Oui
Nouveau composant, nouvel écran	✔ Oui
Modification d'assets (images)	✔ Oui
Ajout d'un module natif	✘ Non — rebuild requis
Mise à jour SDK Expo	✘ Non — rebuild requis
Changement de permissions	✘ Non — rebuild requis
Mise à jour de React Native	✘ Non — rebuild requis

15 · Bonnes pratiques

Performance — Checklist

```
// ✅ Mémoïser les composants enfants qui reçoivent des callbacks
const Item = React.memo(({ item, onPress }: Props) => <View />, (prev, next) => {
  return prev.item.id === next.item.id && prev.item.titre === next.item.titre;
  // Comparaison custom – retourner true = ne pas re-rendre
});

// ✅ Stabiliser les callbacks passés en props
const handlePress = useCallback((id: string) => { dispatch(select(id)); }, [dispatch]);

// ✅ keyExtractor avec un ID stable
keyExtractor={(item) => item.id} // ✅
keyExtractor={(_, index) => String(index)} // ❌ Cause des re-renders

// ✅ getItemLayout si hauteur fixe
getItemLayout=(_, index) => ({ length: 72, offset: 72 * index, index })

// ✅ useNativeDriver: true dès que possible (opacity, transform uniquement)
Animated.timing(value, { toValue: 1, useNativeDriver: true })

// ✅ Images : spécifier les dimensions
<Image source={{ uri }} style={{ width: 200, height: 200 }} /> // ✅
<Image source={{ uri }} /> // ❌

// ✅ Expo Image pour le cache automatique
import { Image } from 'expo-image';
```

Erreurs fréquentes

Erreur	Cause	Correction
Text strings must be rendered within a <Text>	Texte nu dans JSX	Entourer de <Text>
Boucle infinie dans useEffect	Dépendance manquante ou objet inline	Stabiliser avec useCallback / useMemo
Cannot update state on an unmounted component	setState appelé après démontage	Utiliser un flag actif ou AbortController
Rendu lent de longues listes	ScrollView au lieu de FlatList	Utiliser FlatList ou FlashList
FlatList inside ScrollView	Conflits de scroll	Ne jamais imbriquer FlatList dans ScrollView
Image réseau qui ne s'affiche pas	Dimensions manquantes	Toujours spécifier width et height
Mutation directe du state	state.items.push(...)	Retourner un nouveau tableau/objet
undefined is not an object	Optional chaining manquant	user?.profil?.nom
Flash blanc au démarrage	Splash screen mal configuré	Configurer expo-splash-screen

Erreur	Cause	Correction
Crash Android avec LayoutAnimation	API non activée	UIManager.setLayoutAnimationEnabledExperimental(true)

Sécurité

```
// ✅ Stocker les tokens dans SecureStore
await SecureStore.setItemAsync('access_token', token);
// ❌ Ne jamais utiliser AsyncStorage pour les tokens

// ✅ Variables sensibles via EAS Secrets
const apiUrl = Constants.expoConfig?.extra?.apiUrl;
// ❌ const API_KEY = 'sk-abc...'; – jamais dans le code

// ✅ Valider côté serveur – pas uniquement côté client
// ✅ HTTPS uniquement en production
// ✅ Certificate pinning pour les APIs financières/médicales

// ✅ Chiffrer les données sensibles locales
const storage = new MMKV({ encryptionKey: await getEncryptionKey() });
```

16 · Écosystème & Bibliothèques

Bibliothèques recommandées 2025

Catégorie	Bibliothèque	Usage
UI	React Native Paper	Material Design 3 complet
UI	Gluestack UI v2	Accessible, headless, customisable
Styles	NativeWind 4	Tailwind CSS en React Native
Navigation	Expo Router 4	File-based, recommandé
Navigation	React Navigation 7	Contrôle maximal
Animations	Reanimated 3	Animations 60fps worklets
Animations	Moti	API déclarative simple
Gestes	Gesture Handler 2	Gestes avancés cross-platform
Graphiques	React Native Skia	GPU, dessins, effets
Lottie	lottie-react-native	Animations After Effects
State	Zustand 5	Store global léger
State	Redux Toolkit 2	Enterprise, RTK Query
Données	TanStack Query 5	Cache serveur
Formulaires	React Hook Form 7	Performance, TypeScript
Validation	Zod	Schémas TypeScript-first
Stockage	MMKV	Synchrone, rapide, chiffré

Catégorie	Bibliothèque	Usage
Stockage	AsyncStorage	Compatible Expo Go
Stockage	Expo SecureStore	Données sensibles
Images	Expo Image	Cache, transitions, blurhash
Réseau	Axios	HTTP client avancé
Tests	Jest + Testing Library	Unitaires & intégration
Tests E2E	Maestro	Tests déclaratifs YAML
Monitoring	Sentry	Crash reporting production
Build	EAS Build	Builds cloud iOS/Android
OTA	EAS Update	Mises à jour sans store
Performance	FlashList	FlatList ultra-rapide
IA	react-native-executorch	LLM on-device (Meta)

Commandes utiles — Cheat sheet

```
# Créer un projet
npx create-expo-app@latest MonApp

# Développement
npx expo start                # Démarrer Metro
npx expo start --clear        # Vider le cache Metro
npx expo start --dev-client    # Avec Development Build

# Builds natifs locaux
npx expo run:android
npx expo run:ios

# Expo Doctor — diagnostics
npx expo-doctor                # Vérifier les dépendances

# EAS
eas build --platform android --profile preview
eas update --branch production --message "Fix"
eas submit --platform ios

# Tests
npx jest                      # Lancer les tests
npx jest --watch              # Mode watch
npx jest --coverage           # Avec rapport de couverture
npx jest --testPathPattern=CartePresentation # Fichier spécifique

# Qualité
npx eslint . --ext .tsx,.ts   # Linting
npx tsc --noEmit              # Vérification TypeScript
npx prettier --write "**/*.{ts,tsx}" # Formatage

# Nettoyage
rm -rf node_modules && npm install    # Réinstaller les dépendances
watchman watch-del-all             # Vider le cache Watchman (macOS)
npx expo start --clear              # Vider le cache Metro
cd ios && pod install && cd ..         # Réinstaller les pods iOS
```